Promote, Don't Presume: Preregistered Live Testing of Governed LLM Routing Policies

Om Rajesh Pawar and Ahmad Ibrahim

School of Computer Science, University of Birmingham Dubai, UAE
ompawarcloud@gmail.com, a.ibrahim@bham.ac.uk

Abstract. Serving LLM requests on a local GPU and overflowing to a paid cloud API cuts inference cost, but only if the routing policy keeps the latency service-level objective (SLO) without exhausting the cloud budget. Current routers either learn from answer-quality labels that an overflow service does not have, or are tuned once and frozen with no evidence required to replace them. We contribute a governed routing architecture that treats the policy as a service artifact: sealed at runtime, bound to a machine-readable agreement that fixes an SLO guarantee and a cloud-call quota, and replaced only when a candidate clears five promotion gates. We evaluate three policies (the promoted candidate, the incumbent it replaced, and an ungoverned router) in 74 live runs on real hardware, with `llama3.1:8b` serving locally and the production Gemini 2.5 Flash API as overflow, under nominal load, injected outages, and workload drift. Under an agreement allowing at most 10% SLO violations and 30% cloud calls, the governed policy meets the SLO guarantee in every nominal run while sending 12.7% of requests to the cloud, a 5.6 times lower cloud cost per SLO-compliant request than the ungoverned router, which meets the SLO only by sending 82% of requests to the cloud. A prompt-only router, the signal that quality-based routers use, needs a 70% quota to meet the SLO at all. Because the policy is a sealed artifact rather than code tied to one machine, the same governance applies to any AI service that mixes owned and rented capacity.

Keywords: LLM serving · Cloud-edge continuum · SLO · Service governance · Cost-efficient orchestration · SLA-aware routing

1 Introduction

Cloud LLM APIs bill every token, and at enterprise scale the bill already changes behaviour: Microsoft is withdrawing most Claude Code licences in one division, and Uber's CTO reported spending the company's 2026 AI coding budget in four months [1]. A local GPU serves the same kind of request at near-zero marginal cost, but it runs only a few requests at once. Under bursty traffic requests queue, and the time to first token (TTFT) breaks its SLO long before mean latency moves. The economical design is therefore local-first serving with a paid cloud API as overflow. Every request becomes a placement decision on the cloud-edge continuum: keep it local for free, or pay to offload it before it breaks the SLO.

The routing policy decides whether the saving survives, and current practice fails in two ways. A threshold router that offloads whenever the queue is long, with no budget, keeps the SLO by sending most traffic to the cloud, so the bill returns. Learned routers such as RouteLLM [2] and CARROT [4] decide from the prompt, trained on labels of which model answers better. In an overflow service both tiers answer the same request, there is no quality label, and the prompt says little about whether the GPU queue is about to breach the SLO. Whichever policy is chosen is then frozen, and replacing it is a code change with no evidence requirement, so a policy that looks better in testing can break the service agreement once deployed.

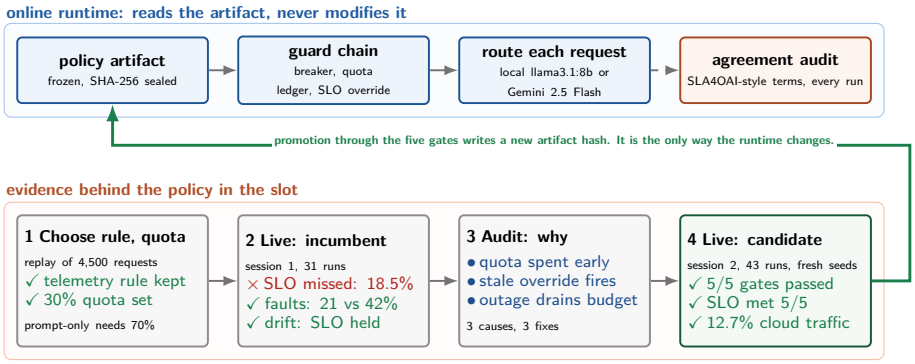


Fig. 1. Governed routing. Top: the runtime reads a sealed policy artifact and never modifies it; an auditor checks every run against the agreement. Bottom: the evidence behind the policy now in the slot (✓ pass, × fail, • finding). Promotion through the five gates of Table 1 is the only way to write a new artifact.

We manage the policy the way service-oriented systems manage any contract-bearing component (Fig. 1). It is versioned and hash-sealed, bound to an agreement that states the SLO guarantee and a cloud-call quota, protected by a guard chain it cannot override, and replaced only when a candidate clears five promotion gates. We test this in 74 audited live runs against Gemini 2.5 Flash, 71 of them preregistered, including injected outages, latency spikes, and workload drift. We contribute:

1. **An agreement-bound routing architecture**: a sealed policy behind a guard chain, bound to an SLA4OAI-style agreement [15] that an auditor checks on every run (Sect. 3).
2. **An evidence-gated promotion protocol**: five preregistered gates evaluated at equal cloud budgets on held-out seeds (Sects. 3.3 and 4).
3. **Live evidence against baselines under the same constraints**: the promoted policy meets the agreement on 12.7% cloud traffic, where ungoverned

and prompt-only routers cannot, and stays available under provider faults (Sect. 5). Code, frozen artifacts, agreements, and protocols are released.[1]

Together, these show that a local-first LLM service can meet its SLO at a fraction of the cloud cost once its routing policy is governed as a service artifact, on any hardware the policy is calibrated for.

2 Related Work

LLM routing. RouteLLM [2], RouterBench [3], and CARROT [4] learn or benchmark routing across model tiers to trade cost against answer quality, and LLMRouterBench [5] finds that many routers are not statistically better than simple baselines. All of them need per-request quality labels. Lodestar [6] and OrcaRouter [7] adapt routing online or from offline learning, which makes the policy itself a moving part and raises the question we address: what evidence a change needs.

Orchestration across the continuum. PerLLM [8], ConsRoute [9], and Yu et al. [10] place LLM inference across edge and cloud, and AIF-Router [11] routes across heterogeneous edge tiers with active inference. Sedlak et al. [12] learn causal SLO models on edge devices, DeepSLOs [13] structure SLOs across stakeholders, and Lapkovskis et al. [14] benchmark dynamic SLO compliance. These systems make the placement decision adaptive; we govern how the decision itself may change.

Agreements, governance, and resilience. SLA4OAI [15] attaches metrics, guarantees, and quotas to API descriptions, Governify [16] audits services against such agreements, and Pricing4SaaS [17] treats a pricing model as a versioned artifact that drives operation. MLOps promotes models through staged pipelines [18], usually gated on offline accuracy, and AgentChaos [19] injects LLM API faults to test agents. No prior work binds a service's routing policy to an agreement whose promotion requires preregistered, budget-matched SLO evidence, or tests such a policy live under injected provider faults.

[1] `https://github.com/OmRajeshPawar/Governed-LLM-Routing-Policy`

3 Governed Routing Architecture

3.1 The Agreement: What the Service Promises

Every design decision is judged against what the service promises. The *SLO* is per request: TTFT at most 3.84 s, the 65th percentile of all-local TTFT, which an all-local service breaks on about a third of requests. A request that exceeds the SLO, or fails with a provider error or timeout, is an *SLO violation*; failed requests are also counted on their own. The *SLO guarantee* is per run: at most 10% of a run's requests violate the SLO. The *quota* caps cloud calls at 30% of a run's requests, which bounds the cloud bill (Sect. 4 explains both choices).

These terms live in a machine-readable *agreement* modelled on SLA4OAI [15] (Fig. 2), with governance terms: the hash of the only policy allowed to run, the breaker rule, and the promotion gates. An auditor replays each run's event log against the agreement and reports every term as met or violated. Because the quota is a versioned term rather than a constant in code, a renegotiated agreement can re-judge existing logs without a code change. We report cost as cloud USD per SLO-compliant request, which prices only useful work.

```
plans:
  local_first:
    guarantees:
      ttft_slo: {threshold_ms: 3839.5, max_violation_rate: 0.10}
    quotas:
      cloud_calls: {max_fraction_of_episode: 0.30, enforcement: hard,
        counts: billable}      # breaker probes exempt, failed calls refunded
governance:
  artifact: {sha256: d3b48bfa...dcfb}   # the only policy allowed to run
  breaker: {rule: no cloud dispatch while open, close_after_probes: 3,
    open_on: {consecutive_failures: 3, latency_x_frozen_p95: 2.0}}
  promotion_bar: {seed_support: ">=4/5", slo_bound: "<=+1.0 pt",
    tail_gain: "P95 ratio <=0.95", no_regression: "<=5 pt", decision_ms: 5}
```

Fig. 2. Excerpt of the routing agreement. The full file is in the repository.

3.2 Runtime: A Sealed Policy Behind a Guard Chain

The routing policy is a versioned *artifact*: for each load level (low, medium, high, or burst, recognised from queue and arrival features) it names the scoring rule and threshold to use, and its SHA-256 hash is recorded in every run manifest. The runtime reads the artifact and never writes it, so every run is traceable to one tested policy.

Hard constraints are enforced independently of the policy. Before the scoring rule is consulted, a guard chain checks, in a fixed order, whether both tiers are degraded, whether the cloud circuit breaker is open, and whether the local tier is in hard breach (queue past a cap, or rolling P95 TTFT above the SLO). Any cloud call then needs room in the quota. The breaker opens after three consecutive cloud failures, or when cloud latency exceeds twice its calibrated P95, and closes after three clean probe calls. The whole chain adds microseconds per decision.

3.3 Policies and Promotion Gates

Three policies appear in the results. The *incumbent* is the policy in the slot before promotion. Its scoring rule is a telemetry heuristic: a weighted sum of seven normalised pre-decision features (queue depth, active local requests, rolling P95 TTFT over the SLO, rolling P95 service time over its target, burst ratio, VRAM use, and estimated request work), which offloads when the sum passes a threshold. The weights and the threshold are tuned on the replay corpus of Sect. 4 and stored in the artifact. The *ungoverned* router runs the same rule and threshold with no breaker and no quota; it stands for the common practice of threshold overflow without a budget. The *candidate* is the proposed replacement (Sect. 5.1). It may replace the incumbent only by clearing all five gates of Table 1, each of which targets one way a replacement can degrade the service.

Table 1. The five promotion gates and the reason for each. The last column is the promoted candidate's result against the incumbent on five held-out seeds (Sect. 5.2).

Gate	Pass criterion	Why it exists	**Candidate**
G1 Seed support	At least as good on SLO violation in $\geq$4 of 5 seeds	A gain must hold across traces, not come from one lucky seed	**5 of 5** ✓
G2 SLO bound	Upper 95% CI bound of the violation difference $\leq +1.0$ pt	Statistical evidence that the agreement's main term is not worse	**bound** -0.73 ✓
G3 Tail gain	P95 TTFT ratio ≤ 0.95, CI upper bound < 1.00	SLOs break in the tail; a swap without tail gain is churn	**bound 0.96** ✓
G4 No regression	No seed $\times$ workload cell more than 5 pt worse	An average can hide one workload that gets much worse	**none worse** ✓
G5 Decision cap	At most 5 ms added routing time per request	A signal that costs time can erase the latency it saves	**+0.006 ms** ✓

4 Experimental Setup

Testbed. The local tier runs `llama3.1:8b` through Ollama [21] on an RTX 5060 laptop GPU (8 GB), two requests at a time, so bursts queue for real. The cloud tier is Gemini 2.5 Flash on Vertex AI, streaming, with thinking disabled. A load generator replays seeded request traces of 300 requests whose arrivals are Poisson with the rate alternating between 1.5 and 0.3 requests per second in a 24-request cycle, six of them at the high rate. Every run logs each routing decision, TTFT, breaker transition, and cloud charge.

Conditions. *Nominal* runs have bursty load and healthy tiers. *Fault* runs inject a cloud outage on requests 96–155 and a cloud latency spike on requests 192–251. *Drift* runs (200 requests) change the workload over five phases and slow the cloud in the last one.

Two sessions. Session 1 tests the incumbent against the ungoverned router under all three conditions, to find whether and why the incumbent misses the agreement: 16 nominal runs (8 seeds $\times$ 2 policies), 6 fault and 6 drift runs (3 seeds $\times$ 2 policies), and 3 exploratory runs of a preliminary variant, the only

runs in the study outside a preregistered protocol. Session 2 tests the candidate on five seeds never used before: 15 nominal and 15 fault runs (5 seeds × 3 policies), 5 drift runs of the candidate, and 8 runs that extend session 1's fault and drift blocks from three to five seeds for the other two policies, which is where their five-seed drift figures come from. Because the sessions use different seeds, absolute rates differ between them; every comparison is made within a session.

Choice of scoring rule and quota. A replay of 4,500 recorded requests selects the scoring rule and the quota (Table 2, Fig. 3a); all results in Sect. 5 are measured live. Every replayed request was served locally with its true TTFT, because a routed log hides the breaches its router prevented: for the same requests it reads a local breach rate of 9.3% against the true 35.0%, the selective-labels problem [20]. At a 30% quota the telemetry heuristic violates the SLO on 6.5% of requests, and a classifier trained on the same telemetry (calibrated logistic regression and gradient boosting, the better Brier score on a 70/30 chronological split) ties it at 6.9%. A prompt-only router, representing the signal class of RouteLLM and CARROT, violates on 19.3% and reaches the 10% target only at a 70% quota, at 2.3 times the cloud spend. The heuristic became the incumbent's rule, and 30%, the smallest quota at which the target is reachable, became the agreement's quota.

Table 2. Choosing the scoring rule and the quota: replay of 4,500 recorded requests at equal cloud budgets. *Viol.*: SLO violation at a 30% quota. *Min. quota*: smallest quota that reaches the 10% target. *Cost*: cloud spend at that quota as a share of sending everything to the cloud. ↓ lower is better. Blue: the rule both governed policies use.

Router	Signal used	Viol. %↓	Target	Min. quota↓	Cost↓
Always local	none	35.0	no	–	0.00
Always cloud	none	0.0	yes	100%	1.00
Prompt-only (RouteLLM-style)	prompt features	19.3	no	70%	0.87
Queue threshold	queue length	7.9	yes	30%	0.34
Trained classifier	34 telemetry features	6.9	yes	30%	0.38
Telemetry heuristic	**queue, recent latency**	**6.5**	**yes**	**30%**	**0.37**

5 Results

5.1 Session 1: The Incumbent Misses the SLO

The incumbent held every other term but missed the SLO. On eight preregistered seeds it violated the SLO on 18.5% of requests against 4.7% for the ungoverned router (95% CI of the difference $[+4.8, +25.6]$ points), while honouring its quota, artifact hash, and breaker rule in every run. The audit of its logs showed three causes, and each became one change in the candidate.

Cause 1: stale evidence spent the budget. Almost all of the incumbent's cloud calls came from the hard-breach override. Its rolling-P95 signal stayed high while requests were being offloaded, because offloaded requests never refreshed it, so the quota ran out between requests 96 and 144 and every later request

queued locally. *Fix, fresh-evidence override:* the override applies only while the local tier is busy.

Cause 2: early bursts drained the quota. Nothing stopped the first bursts from taking the whole run's budget. *Fix, paced release:* after t of n arrivals at most $\min(0.3n, \lfloor 0.3t \rfloor + \lceil 0.05n \rceil)$ cloud calls may have been made: the quota is released in step with traffic, after a head start of 5% of the run (15 calls at $n = 300$, which covers the first burst before proportional release accumulates).

Cause 3: outages consumed the recovery budget. In the fault runs, failed calls and breaker probes counted against the quota, so an outage used up the calls the breaker needed to confirm recovery. *Fix, recovery-aware accounting:* probes are exempt and failed calls are refunded (the `counts: billable` term in Fig. 2).

5.2 Session 2: The Candidate Is Promoted

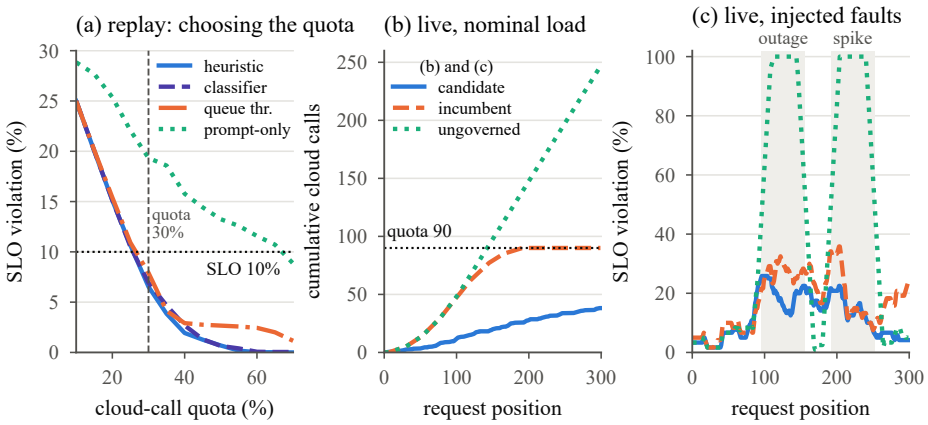


Fig. 3. (a) Replay used to choose the rule and quota: SLO violation against the cloud-call quota (heuristic and classifier overlap). (b) Session 2, nominal load, mean of five seeds: cumulative cloud calls against the 90-call quota. (c) The same policies under the injected outage and latency spike, SLO violation over a 24-request window. Our candidate is the solid blue line.

Table 3. Session 2, five fresh seeds per policy, live Gemini. ↓ lower is better, ↑ higher is better. *Cloud*: share of requests sent to the cloud; the ungoverned router's low violation rate is bought with 82% cloud traffic, which breaks the 30% quota in every run. *Cost*: cloud USD per 10,000 SLO-compliant requests. *Quota met*, *SLO met*: runs out of five that met the quota and the 10% SLO guarantee.

Condition	Policy	Viol. % ↓	P95 TTFT (s) ↓	Cloud % ↓	Cost ↓	Quota met ↑	SLO met ↑
Nominal	**Candidate**	**6.5**	**4.2**	**12.7**	**0.41**	**5/5**	**5/5**
	Incumbent	12.5	6.9	30.0	0.91	5/5	4/5
	Ungoverned	3.7	3.0	82.2	2.29	0/5	5/5
Faults	**Candidate**	**12.2**	**5.9**	**8.6**	**0.24**	**5/5**	**1/5**
	Incumbent	17.3	8.1	29.5	0.87	5/5	0/5
	Ungoverned	42.6	9.5	82.2	2.96	0/5	0/5

The candidate clears all five gates. Against the incumbent on the five fresh seeds it is at least as good on every seed, lowers SLO violation by 6.0 points (CI upper bound −0.73), cuts P95 TTFT to 0.61 of the incumbent's (bound 0.96), and adds 0.006 ms per decision (Table 1, last column). It is the only policy in the study that cleared the gates.

It is the only policy that meets both terms of the agreement. The candidate met the SLO guarantee and the quota in all five nominal runs while sending 12.7% of requests to the cloud (Table 3). Fig. 3b shows why: the incumbent hits its 90-call quota around request 185 and queues locally from then on, while paced release keeps the candidate near 38 calls. The ungoverned router violates the SLO 2.8 points less often, but only by breaking the quota in every run at 5.6 times the candidate's cloud cost per compliant request. A renegotiated 20% quota, re-audited on the same logs without a code change, leaves the candidate compliant in 13 of its 15 runs and the incumbent in none.

5.3 Resilience Under Faults and Drift

Under injected faults, governance keeps the service up. The ungoverned router keeps sending traffic into the outage and the spike and reaches 100% violation in both windows (Fig. 3c). Over the run it fails 301 requests against 32 for the candidate and 39 for the incumbent, and violates the SLO on 42.6% of requests against 12.2% and 17.3% (Table 3). The breaker drives this: once it opens, no request goes to the broken provider. The incumbent, with the same breaker, also stays far below the ungoverned router, so the resilience comes from governance itself, and the candidate is the only policy to meet the 10% guarantee in a fault run.

Under drift, both governed policies protect the service. When the final phase slows the cloud, the ungoverned router sends every request to it and violates the SLO on all of them. The incumbent holds that phase at 13% and the candidate at 15%, a tie, and the candidate does so with no request denied for budget: routing and the breaker carry the phase, not an exhausted quota.

6 Discussion

The gates worked in both directions. The same five gates that promoted the candidate had first rejected the incumbent, a policy that looked sound in replay and missed the SLO live. A selection based on predictive accuracy would have picked the trained classifier instead, which ties the heuristic in replay and fails the gates. What decides promotion for an AI service is therefore the agreement's terms measured live, not a model score.

Quota design matters as much as quota size. Machine-readable terms let the audit expose, run by run, an SLO guarantee and a quota that a policy cannot meet together; the three fixes show that how a quota is released, what evidence may spend it, and what it counts decide whether both terms hold.

Limitations and future work. All live measurements come from a single machine (Intel Core i7-13650HX, 32 GB RAM, NVIDIA RTX 5060 Laptop GPU with 8 GB VRAM) and one Gemini regions, so absolute latencies and thresholds may not transfer but the architecture, agreement, and protocol do, and a second GPU and provider are the next test. Violations arise from queueing at concurrency two, so a larger GPU or a batching server moves the load at which the quota binds rather than the need to govern the policy. Workloads are generated from seeded mixes rather than production traces. Session 2 uses five seeds per policy, so promotion rests on the interval bound and seed-level gates, and it holds for this setup. The candidate was designed from session-1 findings and judged on held-out seeds. Answer quality evaluation was outside the experimental scope: local-first overflow assumes a service contract where the local model meets baseline task acceptance criteria for nominal queries, isolating the orchestration challenge to latency SLO compliance and budget enforcement. Integrating quality verification (e.g., LLM- as-a-judge or reward model gating) into the guard chain is an orthogonal extension left to future work, alongside validation across heterogeneous accelerators and alternative cloud providers.

7 Conclusion

Local-first serving with cloud overflow removes most of the cloud LLM bill, provided the routing policy keeps the latency SLO inside a cloud budget. We made that policy a governed service artifact: sealed at runtime, bound to an agreement that machines can check, protected by a guard chain, and replaced only on preregistered evidence at equal cloud budgets.

The experiments confirmed three things. First, under a 10% SLO guarantee and a 30% quota on cloud calls, the promoted policy met both terms in every nominal run on 12.7% cloud traffic, where an ungoverned router needs 82% of traffic at 5.6 times the cost per compliant request and a prompt-only router needs a 70% quota. Second, the gates work in both directions: they rejected an incumbent that missed the SLO on 18.5% of requests live, and promoted a candidate with half the incumbent's violations on held-out seeds (6.5% against 12.5%). Third, governance keeps the service available under provider faults, with 32 failed requests against 301 for the ungoverned router.

Any AI service that mixes owned and rented capacity, from agent tool calls to multi-model pipelines and edge fleets, has a decision policy whose mistakes show up as broken agreements or surprise bills. Treating that policy as an artifact bound to a contract, which changes only on preregistered evidence, gives operators an auditable path to cheaper AI services. As future work, we will make the system more robust by experiments on various hardware and providers and extend the auditor into a runtime monitor

Acknowledgments. Prompt text in the synthetic traces was produced with LLM assistance (Claude Sonnet 5); every measurement is the real model's response to those prompts. An LLM also assisted with code, debugging, and drafting this manuscript; the authors verified all content and take full responsibility for it.

References

1. Stanciuc, A.M.: Microsoft's quiet Claude Code retreat and the real cost of enterprise AI. The Next Web (25 May 2026). `https://thenextweb.com/news/microsoft-claude-code-retreat-ai-cost`
2. Ong, I., Almahairi, A., Wu, V., Chiang, W.L., Wu, T., Gonzalez, J.E., Kadous, M.W., Stoica, I.: RouteLLM: Learning to route LLMs with preference data. arXiv:2406.18665 (2024)
3. Hu, Q.J., Bieker, J., Li, X., Jiang, N., Keigwin, B., Ranganath, G., Keutzer, K., Upadhyay, S.K.: RouterBench: A benchmark for multi-LLM routing system. arXiv:2403.12031 (2024)
4. Somerstep, S., Maia Polo, F., Melo de Oliveira, A.F., Mangal, P., Silva, M., Bhardwaj, O., Yurochkin, M., Maity, S.: CARROT: A cost aware rate optimal router. arXiv:2502.03261 (2025)
5. Li, H., Zhang, Y., Guo, Z., Wang, C., Tang, S., Zhang, Q., Chen, Y., Qi, B., Ye, P., Bai, L., Wang, Z., Hu, S.: LLMRouterBench: A massive benchmark and unified framework for LLM routing. arXiv:2601.07206 (2026)

6. Lim, G., Zhao, W., Godfrey, B., Shan, J., Xu, L., Xie, L.: Lodestar: An online-learning LLM inference router. arXiv:2606.00946 (2026)
7. Bao, Z., Tian, F., Zhang, C., Chen, Z., Ma, X., Shi, Y.: OrcaRouter: A production-oriented LLM router with hybrid offline-online learning. arXiv:2605.30736 (2026)
8. Yang, Z., Yang, Y., Zhao, C., Guo, Q., He, W., Ji, W.: PerLLM: Personalized inference scheduling with edge-cloud collaboration for diverse LLM services. arXiv:2405.14636 (2024)
9. Qiao, H., Zhang, H., Mao, S., Cheng, S., Liu, J.: ConsRoute: Consistency-aware adaptive query routing for cloud-edge-device large language models. arXiv:2603.21237 (2026)
10. Yu, S., Goudarzi, M., Toosi, A.N.: Efficient routing of inference requests across LLM instances in cloud-edge computing. arXiv:2507.15553 (2025)
11. Wang, Z., Sedlak, B., Dustdar, S.: Active inference-based adaptive routing for heterogeneous edge AI services. arXiv:2604.17373 (2026)
12. Sedlak, B., Casamayor Pujol, V., Donta, P.K., Dustdar, S.: Equilibrium in the computing continuum through active inference. Future Generation Computer Systems **160**, 92–108 (2024). https://doi.org/10.1016/j.future.2024.05.056
13. Casamayor Pujol, V., Sedlak, B., Xu, Y., Donta, P.K., Dustdar, S.: Invited paper: DeepSLOs for the computing continuum. In: Proceedings of the 2024 Workshop on Advanced Tools, Programming Languages, and PLatforms for Implementing and Evaluating Algorithms for Distributed Systems (ApPLIED). ACM (2024). https://doi.org/10.1145/3663338.3663681
14. Lapkovskis, A., Sedlak, B., Magnússon, S., Dustdar, S., Donta, P.K.: Benchmarking dynamic SLO compliance in distributed computing continuum systems. arXiv:2503.03274 (2025)
15. Gamez-Diaz, A., Fernandez, P., Ruiz-Cortes, A.: Automating SLA-driven API development with SLA4OAI. In: Service-Oriented Computing (ICSOC 2019). LNCS, vol. 11895, pp. 20–35. Springer (2019). https://doi.org/10.1007/978-3-030-33702-5_2
16. Fresno-Aranda, R., Ojeda-Perez, J.S., Fernandez, P., Ruiz-Cortes, A.: Governify. An agreement-based service governance framework. Software Impacts **19**, 100629 (2024). https://doi.org/10.1016/j.simpa.2024.100629
17. García-Fernández, A., Parejo, J.A., Ruiz-Cortés, A.: Pricing4SaaS: Towards a pricing model to drive the operation of SaaS. In: CAiSE Forum 2024. Springer (2024). https://doi.org/10.1007/978-3-031-61000-4_6
18. Kreuzberger, D., Kühl, N., Hirschl, S.: Machine learning operations (MLOps): Overview, definition, and architecture. IEEE Access **11**, 31866–31879 (2023). https://doi.org/10.1109/ACCESS.2023.3262138
19. Tan, G., Sun, Z., Shi, J., Zhang, T., He, Z., Wu, Q., Liang, S., Sun, W., He, J., Chen, P., Zhang, C., Shar, L.K., Lo, D.: AgentChaos: Chaos engineering for agent systems via programmatic fault injection. In: 41st IEEE/ACM International Conference on Automated Software Engineering (ASE) (2026). arXiv:2608.06790
20. De-Arteaga, M., Dubrawski, A., Chouldechova, A.: Learning under selective labels in the presence of expert consistency. In: ICML Workshop on Fairness, Accountability, and Transparency in Machine Learning (FAT/ML) (2018)
21. Ollama: Get up and running with large language models locally. https://ollama.com (2024)